



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное бюджетное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

**Лабораторная работа № 1
по дисциплине «Анализ алгоритмов»**

Тема Алгоритмы Левенштейна и Дамерау-Левенштейна

Студент Пантелеев Василий Максимович

Группа ИУ7-52Б

Преподаватели Строганов Дмитрий Владимирович

Москва, 2024

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Расстояние Левенштейна	4
1.2 Расстояние Дамерау-Левенштейна	5
2 Конструкторская часть	6
2.1 Описание алгоритмов	6
3 Технологическая часть	11
3.1 Средства реализации	11
3.2 Реализация алгоритмов	11
3.3 Функциональные тесты	14
4 Исследовательская часть	15
4.1 Технические характеристики ЭВМ	15
4.2 Сравнение по времени	15
4.3 Сравнение по памяти	17
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20

ВВЕДЕНИЕ

В первой лабораторной работе по анализу алгоритмов рассматривается понятие расстояния Левенштейна. Это расстояние определяет минимальное количество операций вставки, удаления и замены, необходимых для преобразования одной строки в другую. Расстояние Левенштейна находит широкое применение в различных областях, включая обработку естественного языка, системах автоматического исправления, а также в биоинформатике для сравнения последовательностей.

Целью данной лабораторной работы является сравнение алгоритмов нахождения расстояния Левенштейна и Дamerau-Левенштейна. Для достижения этой цели необходимо решить следующие задачи:

- ознакомиться с теоретическими аспектами расстояния Левенштейна и расстояния Дamerau-Левенштейна;
- реализовать рекурсивный алгоритм, рекурсивный алгоритм с мемоизацией, нерекурсивный алгоритм для нахождения расстояния Левенштейна, а также нерекурсивный алгоритм для нахождения расстояния Дamerau-Левенштейна;
- сравнить производительность алгоритмов по затраченному процессорному времени и памяти;
- обосновать полученные результаты и сделать выводы о характеристиках и применимости каждого алгоритма.

1 Аналитическая часть

1.1 Расстояние Левенштейна

Расстояние Левенштейна является мерой различия между двумя строками. Оно определяется как минимальное количество односимвольных операций, необходимых для преобразования одной строки в другую. Эти операции включают вставку буквы, её удаление и замену буквы на другую.

Рекуррентная формула расстояния Левенштейна для двух строки S_1 и S_2 , длиной M и N соответственно:

$$D(i, j) = \begin{cases} 0, & i = 0, j = 0 \\ i, & j = 0, i > 0 \\ j, & i = 0, j > 0 \\ \min \begin{cases} D(i, j - 1) + 1 \\ D(i - 1, j) + 1 \\ D(i - 1, j - 1) + m(S_1[i], S_2[i]) \end{cases} & i > 0, j > 0 \end{cases} \quad (1.1)$$

где функция m определена как:

$$m(a, b) = \begin{cases} 0 & \text{если } a = b, \\ 1 & \text{иначе} \end{cases}. \quad (1.2)$$

Существует несколько возможных реализаций алгоритма для вычисления расстояния Левенштейна.

Рекурсивный алгоритм. Это базовая реализация, использующая рекурсию для вычисления расстояния. Однако она неэффективна для длинных строк из-за множественных рекурсивных вызовов.

Рекурсивный алгоритм с кэшированием. Данная реализация использует технику кэширования для хранения результатов уже выполненных подзадач, что снижает количество рекурсивных вызовов и улучшает производительность.

Нерекурсивный алгоритм, основанный на динамическом программи-

ровании. Этот алгоритм использует двумерный массив для хранения промежуточных результатов и заменяет рекурсию итерациями.

1.2 Расстояние Дамерау-Левенштейна

Расстояние Дамерау-Левенштейна является расширением расстояния Левенштейна и учитывает дополнительно операцию транспозиции двух соседних символов. Расстояние Дамерау-Левенштейна также может быть вычислено с использованием динамического программирования с добавлением дополнительной логики для обработки транспозиции.

Рекуррентная формула расстояния Дамерау-Левенштейна для двух строк S_1 и S_2 , длиной M и N соответственно:

$$D(i, j) = \begin{cases} i, & j = 0, i > 0 \\ j, & i = 0, j > 0 \\ D(i - 1, j - 1), & i > 0, j > 0, S_1[i] = S_2[j] \\ 1 + \min \begin{cases} D(i, j - 1) \\ D(i - 1, j) \\ D(i - 1, j - 1) \end{cases}, & \begin{matrix} S_1[i] = S_2[j - 1], \\ S_1[i - 1] = S_2[j] \end{matrix} \\ 1 + \min \begin{cases} D(i, j - 1) \\ D(i - 1, j) \\ D(i - 1, j - 1) \end{cases}, & \text{иначе} \end{cases} \quad (1.3)$$

Вывод

В данном разделе были рассмотрены алгоритмы нахождения расстояния Левенштейна и Дамерау-Левенштейна, формулы которых задаются рекуррентно, а следовательно, данные алгоритмы могут быть реализованы рекурсивно и итерационно. На вход алгоритмам будут поступать две строки, которые могут содержать как русские, так и английские буквы, также будет предусмотрен ввод пустых строк.

2 Конструкторская часть

В данном разделе будут приведены схемы алгоритмов нахождения расстояний Левенштейна и Дameraу-Левенштейна.

2.1 Описание алгоритмов

На вход алгоритмов подаются 2 строки - str1 и str2, на выходе получается искомое расстояние.

На схемах 2.1 — 2.5 приведены описания алгоритмов нахождения расстояния Левенштейна и Дameraу-Левенштейна:

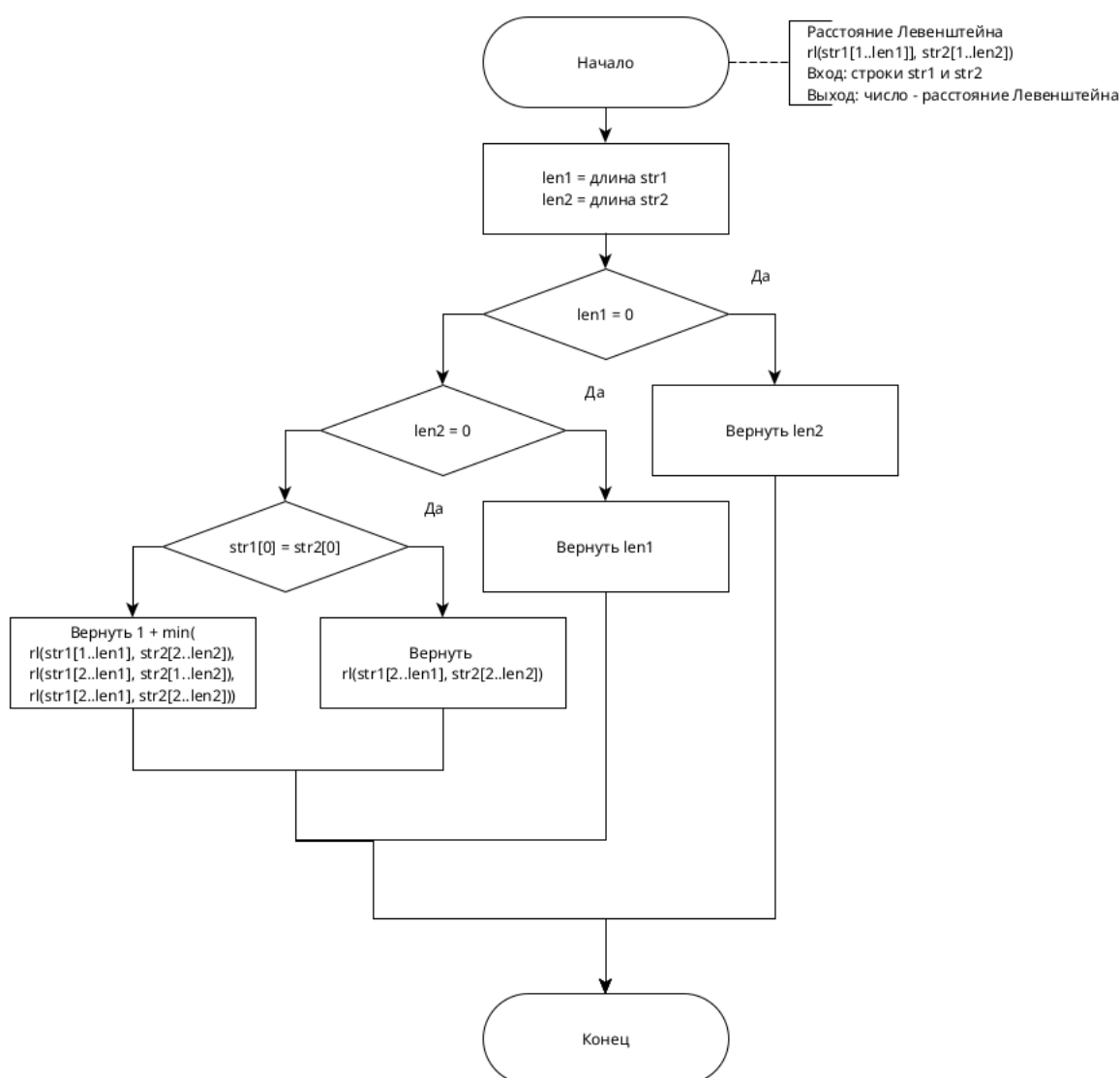


Рисунок 2.1 — Схема рекурсивного алгоритма нахождения расстояния Левенштейна

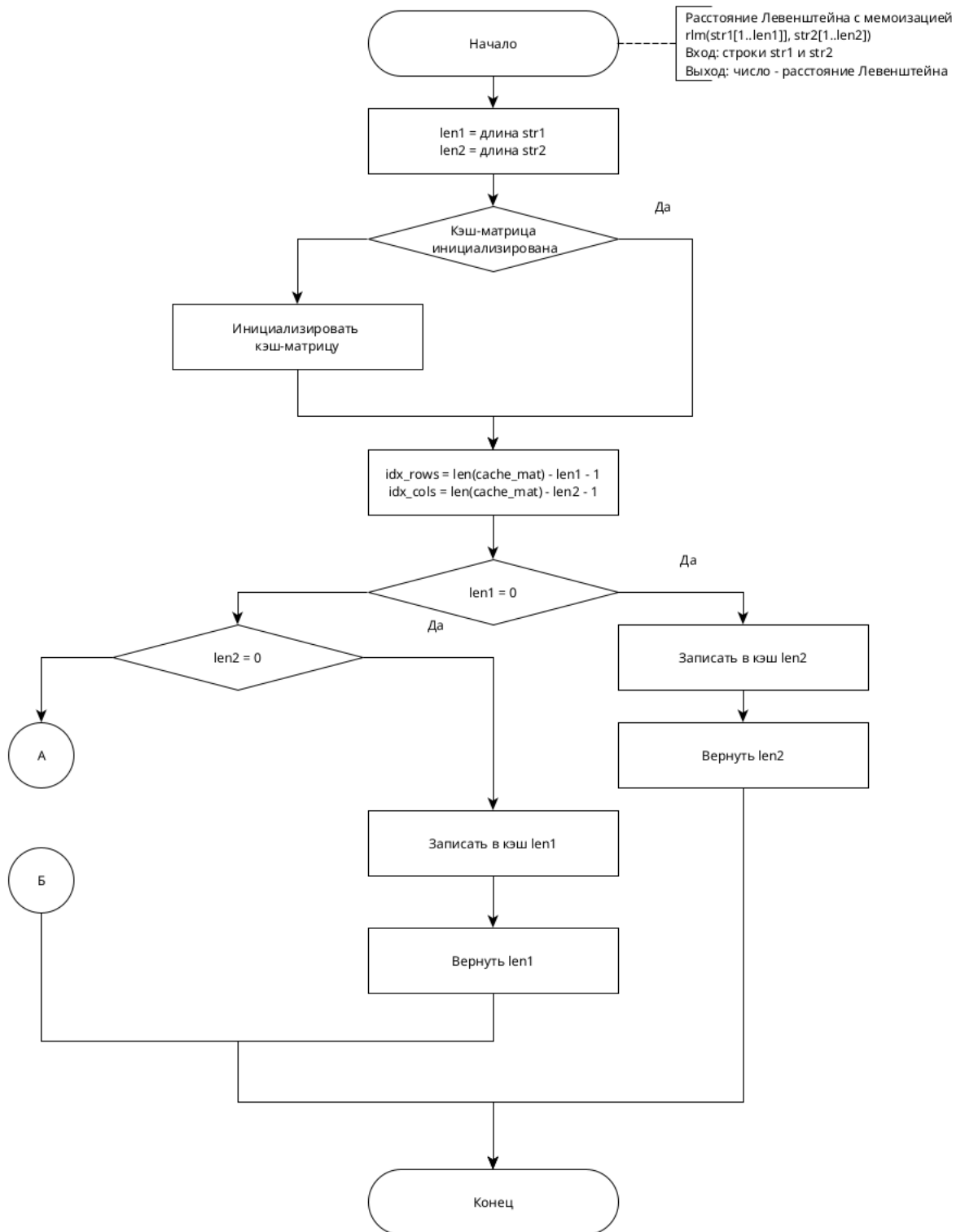


Рисунок 2.2 — Схема рекурсивного алгоритма нахождения расстояния Левенштейна с мемоизацией

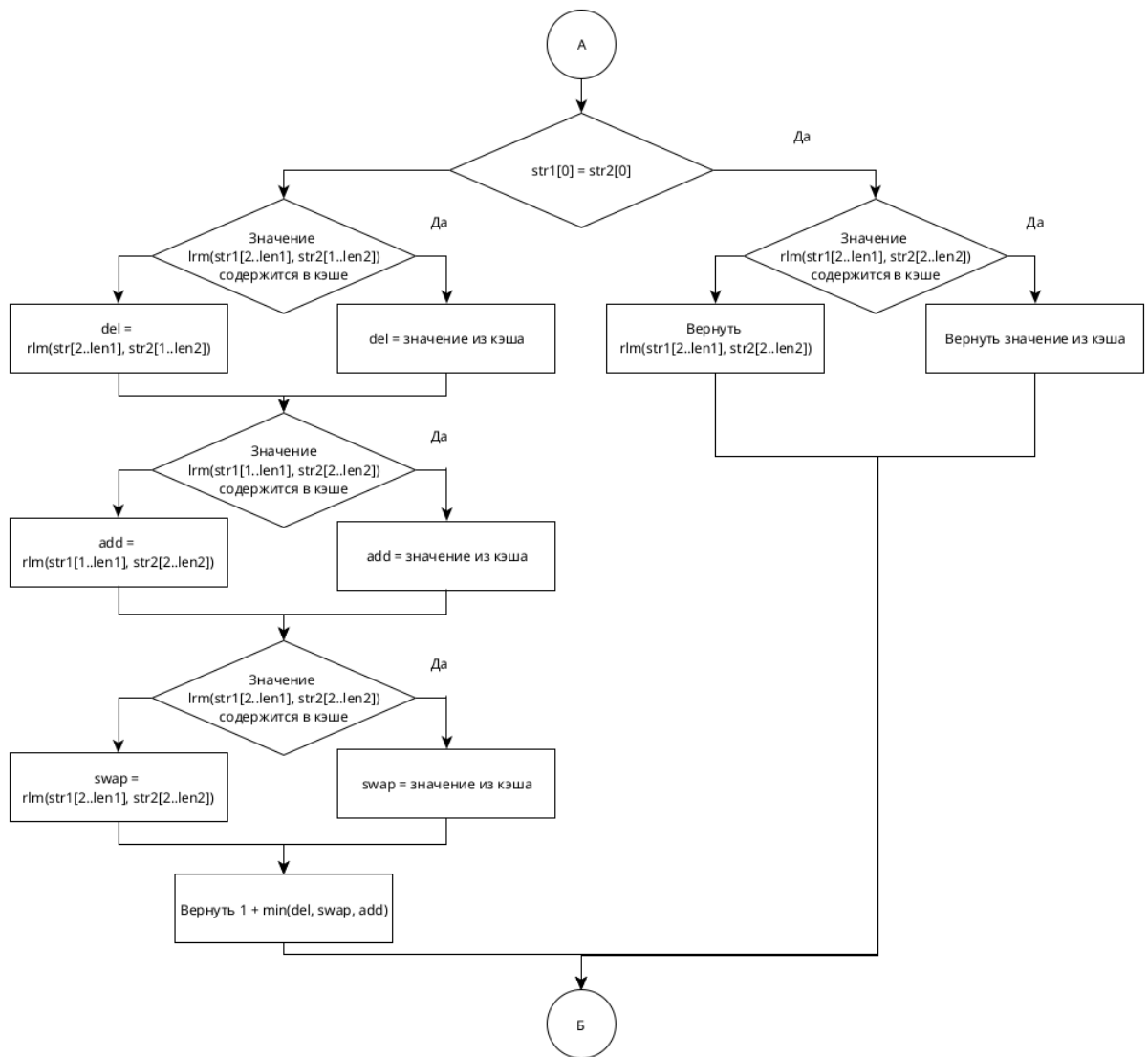


Рисунок 2.3 — Схема рекурсивного алгоритма нахождения расстояния Левенштейна с мемоизацией (продолжение)

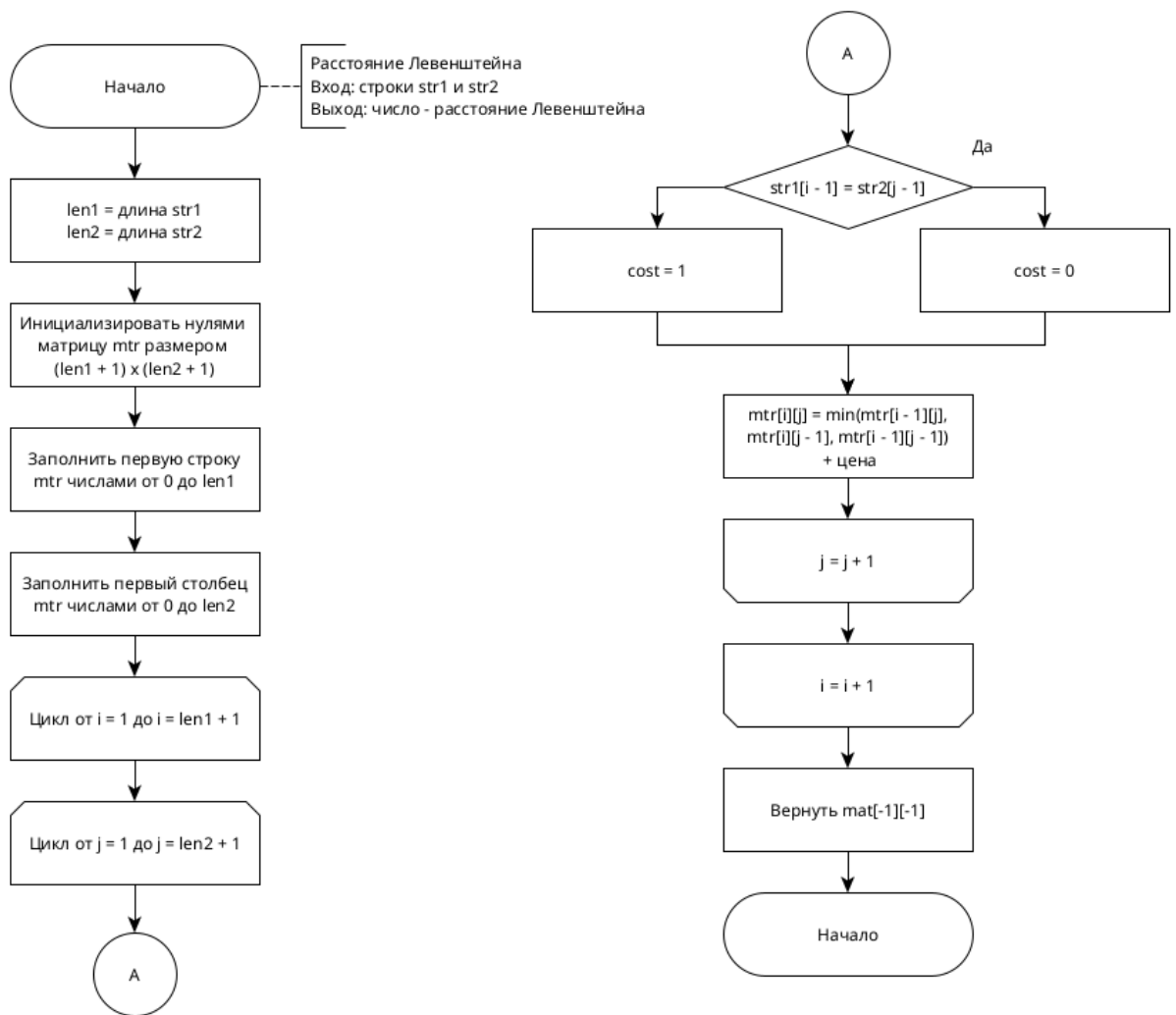


Рисунок 2.4 — Схема нерекурсивного алгоритма нахождения расстояния Левенштейна

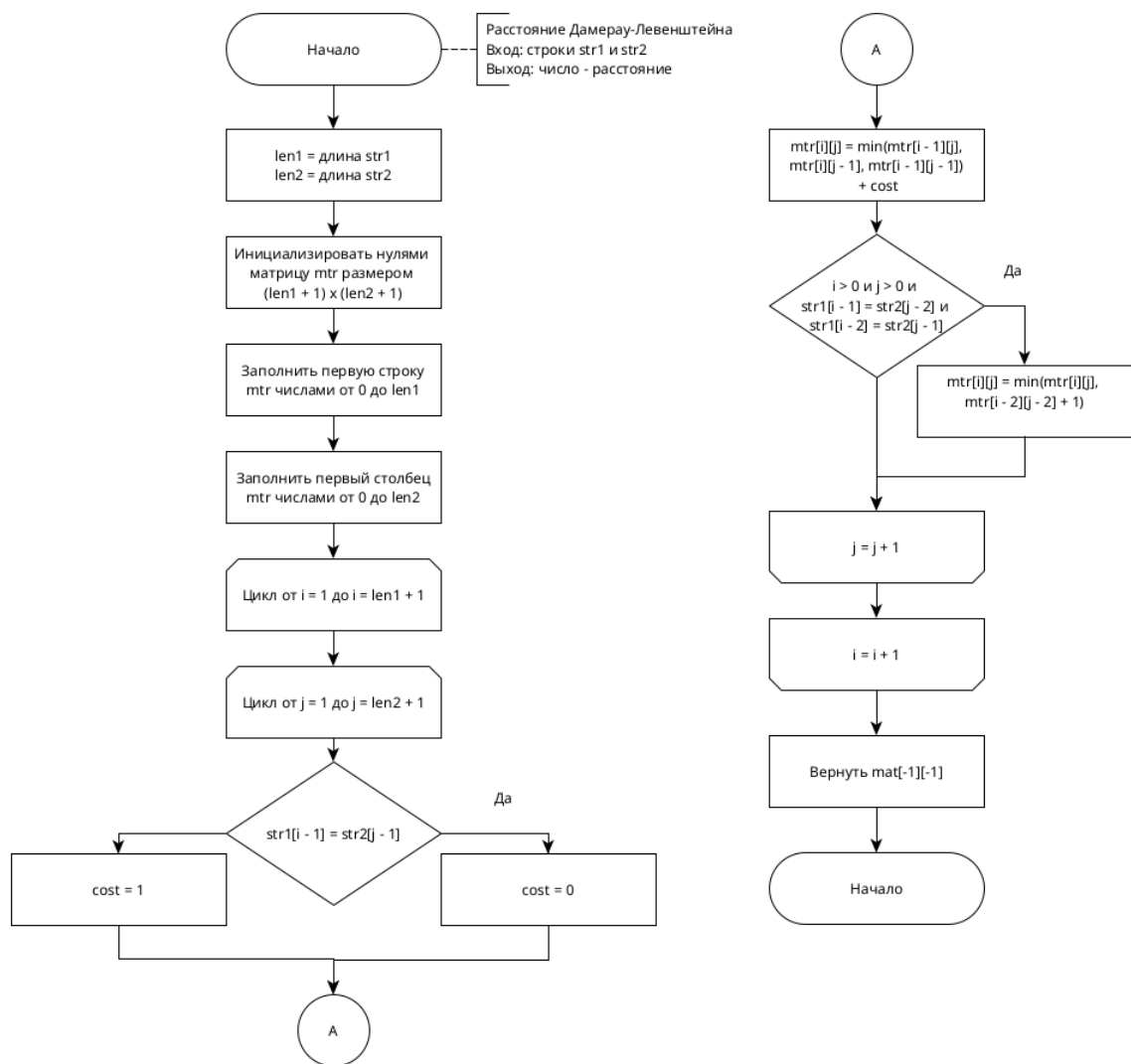


Рисунок 2.5 — Схема нерекурсивного алгоритма нахождения расстояния Дамерау-Левенштейна

Вывод

В данном разделе на основе теоретических аспектов были разработаны и представлены схемы рекурсивного алгоритма, рекурсивного алгоритма с кэшированием, нерекурсивного алгоритма на основе динамического программирования, а также схема алгоритма Дамерау-Левенштейна.

3 Технологическая часть

3.1 Средства реализации

Для реализации алгоритмов в этой лабораторной работы был выбран язык *Python*^[1] по следующим причинам:

- python предоставляет необходимые структуры данных, такие как двумерные списки (массивы);
- поддерживаемые библиотеки, такие как *matplotlib* позволяют визуализировать результаты исследования.

3.2 Реализация алгоритмов

В листингах 3.1 — 3.4 представлены реализации алгоритмов.

Листинг 3.1 — Рекурсивный алгоритм Левенштейна

```
def levenshtein_recursive(str1, str2):
    len1 = len(str1)
    len2 = len(str2)

    if len1 == 0:
        return len2
    elif len2 == 0:
        return len1
    elif str1[0] == str2[0]:
        return levenshtein_recursive(str1[1:], str2[1:])

    return 1 + min(
        levenshtein_recursive(str1[1:], str2),
        levenshtein_recursive(str1, str2[1:]),
        levenshtein_recursive(str1[1:], str2[1:])
    )
```

Листинг 3.2 — Рекурсивный алгоритм Левенштейна с Мемоизацией

```
def levenshtein_recursive_memo(str1, str2, cache_matrix=None):
    len1 = len(str1)
    len2 = len(str2)

    if cache_matrix is None:
        cache_matrix = [[-1] * (len2 + 1) for _ in range(len1 + 1)]

    idx_rows = len(cache_matrix) - len1 - 1
    idx_cols = len(cache_matrix[0]) - len2 - 1
```

```

if len1 == 0:
    cache_matrix[idx_rows][idx_cols] = len2
    return len2
elif len2 == 0:
    cache_matrix[idx_rows][idx_cols] = len1
    return len1
elif str1[0] == str2[0]:
    curr = cache_matrix[idx_rows + 1][idx_cols + 1]
    return curr if curr != -1 else levenshtein_recursive_memo(str1[1:], str2
        [1:], cache_matrix)

delete = cache_matrix[idx_rows + 1][idx_cols]
add = cache_matrix[idx_rows][idx_cols + 1]
swap = cache_matrix[idx_rows + 1][idx_cols + 1]

return 1 + min(
    delete if delete != -1 else levenshtein_recursive_memo(str1[1:], str2,
        cache_matrix),
    add if add != -1 else levenshtein_recursive_memo(str1, str2[1:],
        cache_matrix),
    swap if swap != -1 else levenshtein_recursive_memo(str1[1:], str2[1:],
        cache_matrix)
)

```

Листинг 3.3 — Нерекурсивный алгоритм Левенштейна

```

def levenshtein(str1, str2):
    len1 = len(str1)
    len2 = len(str2)

    cache_matrix = [[0] * (len2 + 1) for _ in range(len1 + 1)]

    for i in range(len1 + 1):
        cache_matrix[i][0] = i
    for j in range(len2 + 1):
        cache_matrix[0][j] = j

    for i in range(1, len1 + 1):
        for j in range(1, len2 + 1):
            cost = 0 if str1[i - 1] == str2[j - 1] else 1

            cache_matrix[i][j] = min(
                cache_matrix[i - 1][j],
                cache_matrix[i][j - 1],
                cache_matrix[i - 1][j - 1]
            ) + cost

```

```
return cache_matrix[-1][-1]
```

Листинг 3.4 — Нерекурсивный алгоритм Дамерау-Левенштейна

```
def damerau_levenshtein(str1, str2):
    len1 = len(str1)
    len2 = len(str2)

    cache_matrix = [[0] * (len2 + 1) for _ in range(len1 + 1)]

    for i in range(len1 + 1):
        cache_matrix[i][0] = i
    for j in range(len2 + 1):
        cache_matrix[0][j] = j

    for i in range(1, len1 + 1):
        for j in range(1, len2 + 1):
            cost = 0 if str1[i - 1] == str2[j - 1] else 1

            cache_matrix[i][j] = min(
                cache_matrix[i - 1][j],
                cache_matrix[i][j - 1],
                cache_matrix[i - 1][j - 1]
            ) + cost

            if i > 1 and j > 1 and str1[i - 1] == str2[j - 2] and str1[i - 2] == str2[j - 1]:
                cache_matrix[i][j] = min(cache_matrix[i][j], cache_matrix[i - 2][j - 2] + 1)

    return cache_matrix[-1][-1]
```

3.3 Функциональные тесты

В таблице 3.1 представлены функциональные тесты для проверки корректности работы программы.

Таблица 3.1 — Функциональные тесты

str_1	str_2	Левенштейн	Дамерау-Левенштейн
«»	«»	0	0
abc	«»	3	3
«»	abc	3	3
abc	abc	0	0
abc	xyz	3	3
abc	abcd	1	1
abcd	abc	1	1
kitten	sitting	3	3
reka	muka	2	2
apple	appel	2	1

Вывод

В технологической части были представлены листинги реализованных алгоритмов, включая рекурсивные и нерекурсивные версии нахождения расстояния Левенштейна и Дамерау-Левенштейна. Также были проведены функциональные тесты, которые подтвердили корректность работы всех алгоритмов на различных входных данных, включая граничные случаи. Результаты тестов показали, что разработанные реализации соответствуют поставленным требованиям.

4 Исследовательская часть

4.1 Технические характеристики ЭВМ

Все замеры проводились на ЭВМ, характеристики которой приведены ниже:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16;
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS;
- оперативная память: 16,0 ГБ.

4.2 Сравнение по времени

Время выполнения работы алгоритмов представлено в секундах. Для повышения точности измерений проводилось 200 замеров. Замеры проводились использованием функции *process_time*^[2] модуля *time* стандартной библиотеки Python, позволяющей измерять процессорное время, потраченное на выполнение алгоритма. Графики построены при помощи средств библиотеки *matplotlib*^[3].

В таблице 4.1 приведено сравнение алгоритмов по времени выполнения на размерах строк от 1 до 10 с шагом 1. На графике 4.1 приведена визуализация результатов.

Таблица 4.1 — Сравнение алгоритмов по времени выполнения

Размер строк	Рекурсивный	С кэшем	Нерекурсивный	Дамерау
1	1×10^{-6}	2×10^{-6}	1×10^{-6}	2×10^{-6}
2	3×10^{-6}	4×10^{-6}	2×10^{-6}	3×10^{-6}
3	1×10^{-5}	1×10^{-5}	4×10^{-6}	4×10^{-6}
4	5×10^{-5}	5×10^{-5}	5×10^{-6}	6×10^{-6}
5	3×10^{-4}	2×10^{-4}	7×10^{-6}	9×10^{-6}
6	1×10^{-3}	9×10^{-4}	9×10^{-6}	1.2×10^{-5}
7	7×10^{-3}	6×10^{-3}	1.3×10^{-5}	1.5×10^{-5}
8	0.04	0.03	1.6×10^{-5}	2.0×10^{-5}
9	0.20	0.16	2.0×10^{-5}	2.4×10^{-5}
10	1.00	0.88	2.4×10^{-5}	3.0×10^{-5}

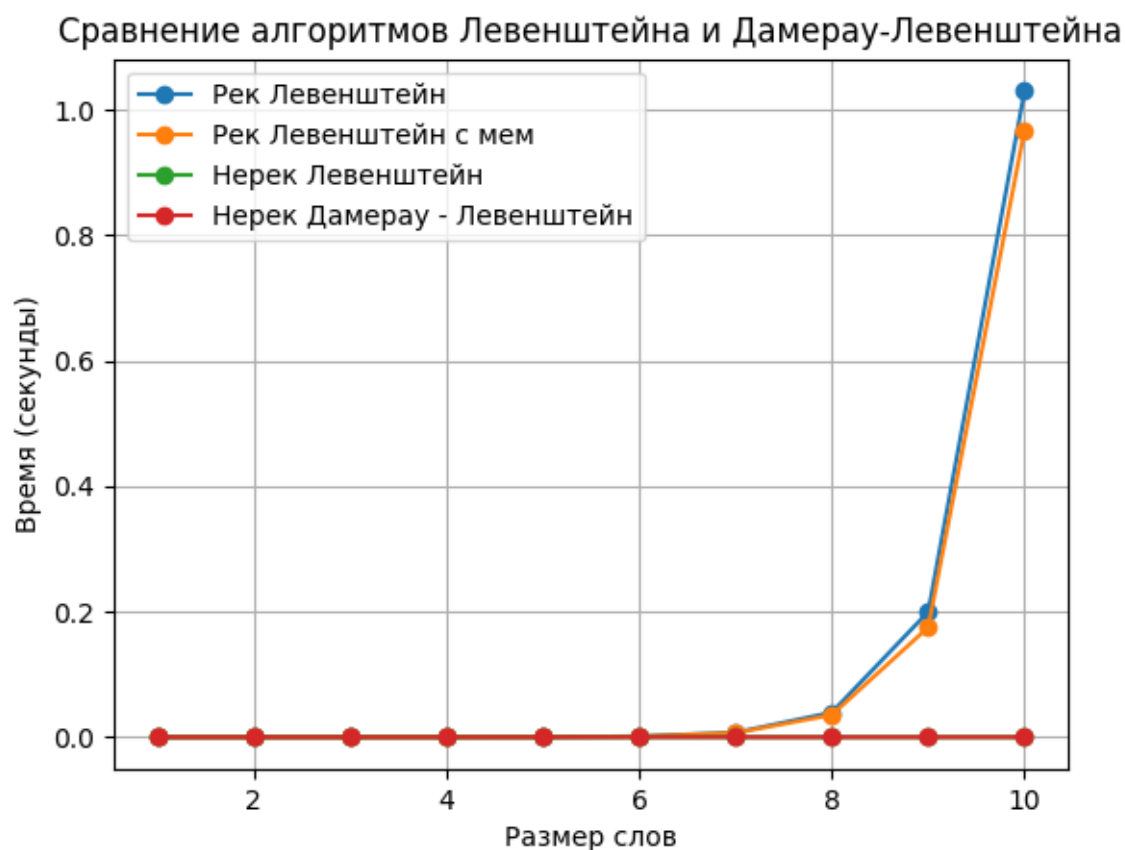


Рисунок 4.1 — Сравнение алгоритмов по времени выполнения

В таблице 4.2 приведено сравнение нерекурсивных алгоритмов по времени выполнения на размерах строк от 10 до 500. На графике 4.2 приведена визуализация результатов.

10

Таблица 4.2 — Сравнение нерекурсивных алгоритмов по времени выполнения

Размер строк	Левенштейн	Дамерау-Левенштейн
10	2.4×10^{-5}	3.0×10^{-5}
20	9.3×10^{-5}	1.1×10^{-4}
30	3.7×10^{-4}	4.5×10^{-4}
50	5.8×10^{-4}	7.2×10^{-4}
100	2.2×10^{-3}	2.8×10^{-3}
150	4.9×10^{-3}	6.3×10^{-3}
200	8.7×10^{-3}	0.01
300	0.02	0.03
400	0.04	0.05
500	0.06	0.08

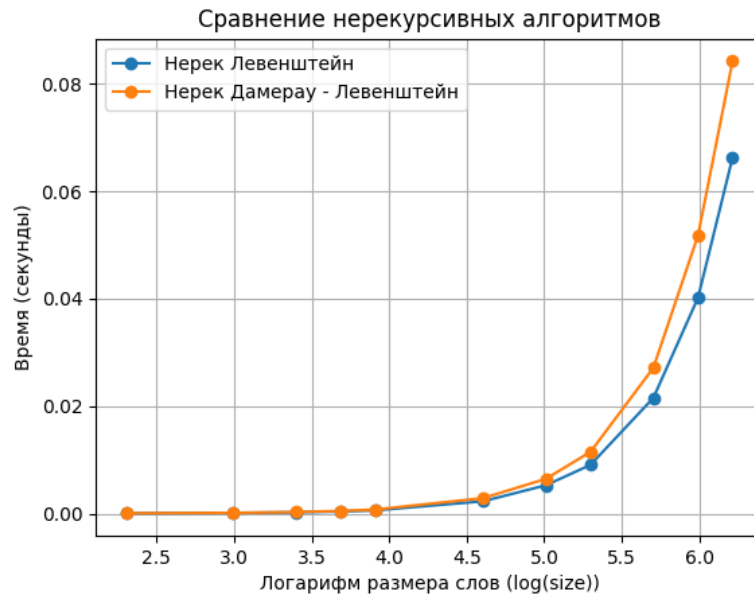


Рисунок 4.2 — Сравнение нерекурсивных алгоритмов по времени выполнения

4.3 Сравнение по памяти

В таблице 4.3 приведено сравнение различных реализаций по памяти. Нерекурсивные алгоритмы Левенштейна и Дамерау-Левенштейна эквивалентны по расходу памяти.

Затраты памяти для алгоритмов:

- рекурсивный алгоритм нахождения расстояния Левенштейна – $\text{len1} + \text{len2}$;
- рекурсивный алгоритм нахождения расстояния Левенштейна с мемоизацией – $\text{len1} * \text{len2} + \text{len1} + \text{len2}$;
- нерекурсивный алгоритм нахождения расстояния Левенштейна – $\text{len1} * \text{len2} + \text{len1} + \text{len2}$;
- нерекурсивный алгоритм нахождения расстояния Дамерау-Левенштейна – $\text{len1} * \text{len2} + \text{len1} + \text{len2}$.

len1 и len2 - длины строк на входе.

Таблица 4.3 — Сравнение по памяти двух строк длины len1 и len2 соответственно

len1	len2	Рекурсивно, байт	Рекурсивно + мем, байт	Нерекурсивно, байт
1	1	20	26	10
10	10	20	161	145
100	100	200	10421	10405
1000	1000	2000	1004021	1004005

Вывод

В исследовательской части было проведено сравнение различных реализаций алгоритмов нахождения расстояния Левенштейна и Дamerau-Левенштейна по времени выполнения и потребляемым ресурсам. Анализ показал, что рекурсивный алгоритм с кэшированием выигрывает по времени примерно в 1.25 раза по сравнению с базовым рекурсивным подходом за счет уменьшения числа повторных вычислений. Однако этот алгоритм требует большего объема памяти, так как хранит промежуточные результаты для всех подзадач.

Рекурсивные алгоритмы имеют высокую временную сложность, но на строках, размером до 6, разница не существенна. В свою очередь, нерекурсивные алгоритмы, основанные на динамическом программировании, демонстрируют лучшую производительность на длинных словах (от 6 символов) за счет более эффективного использования памяти и отсутствия накладных расходов, связанных с рекурсией.

ЗАКЛЮЧЕНИЕ

В ходе работы было установлено, что существует различие в временной эффективности между рекурсивными и нерекурсивными реализациями алгоритмов Левенштейна и Дамерау-Левенштейна. Разработанное программное обеспечение позволило осуществить замеры процессорного времени выполнения для различных длин строк, что подтвердило теоретические ожидания.

Результаты исследования показали, что нерекурсивная матричная реализация алгоритмов превосходит рекурсивные подходы по времени выполнения, особенно при увеличении длины обрабатываемых строк (от 6 символов). Однако следует отметить, что данная матричная реализация требует больше памяти, что может быть критично в условиях ограниченных ресурсов.

Оптимизация рекурсивного алгоритма выигрывает у обычной реализации по времени (в 1.25 раза), но проигрывает по памяти из-за необходимости хранить кэш-матрицу.

Таким образом, выбор между рекурсивной и нерекурсивной реализацией данных алгоритмов зависит от конкретных требований к производительности и ресурсам.

В ходе выполненной работы были выполнены следующие задачи:

- реализованы различные алгоритмы для вычисления расстояния Левенштейна и Дамерау-Левенштейна;
- проведено сравнение алгоритмов по времени выполнения и памяти;
- полученные результаты обоснованы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Welcome to Python – [Электронный ресурс]. URL: <https://https://www.python.org> (дата обращения: 15.09.2024).
2. process_time – [Электронный ресурс]. URL: <https://docs.python.org/3/library/time.html#functions> (Дата обращения 15.09.2024).
3. Библиотека matplotlib – [Электронный ресурс]. URL: <https://matplotlib.org/> (дата обращения: 15.09.2024).